Recursion Analysis for Certifying Parallelizability

Neea Rusch[1] and Nathaniel Young[2]

[1] Uppsala University, Uppsala, Sweden
[2] Cornell University, Ithaca, NY, United States

Abstract

We investigate discovery of parallelization potential in recursive functional programs. The key intuition is to relate parallelizability to membership in the complexity class NC, the class of efficiently parallelizable problems. From this premise, we develop a static program analysis that responds affirmatively when an input program is amenable to parallelization. The theoretical machinery is borrowed from an established implicit characterization of NC. The benefit is that, instead of tracking dataflow over arbitrary program constructs, the analysis determines whether the input program has the right syntactical form to satisfy the membership criterion. In the abstract we focus on algorithmic foundations, but our complete results include a concrete analyzer and empirical experiments. Through these advancements, we can quantify the expressive power and practicality of the analysis. Our preliminary results show that the analysis can automatically recognize many parallelizable programs and is a viable candidate for integration into larger compilation workflows.

1 Parallelizability via NC membership

In complexity theory, the class NC ("Nick's Class") contains problems efficiently solvable on an idealized model of a parallel computer [3, 4]. A problem with input size n is in class NC if it can be solved in polylogarithmic time (poly-$\log n$) using a polynomial (poly-n) number of parallel processors [9, 4]. The main models of computation in which NC is defined are low-depth Boolean circuits [6, Ch. 1], Turing machines with special properties [9, 10], and parallel random-access machines [11]. A challenge with these modeling abstractions is that they are rather distant from how computational instructions are written in practice, i.e., programs. An alternative perspective is to consider implicit syntactic characterizations of complexity classes, expressed via programming languages. Among the benefits, a programming languages-based characterizations enable us to lift results from foundational complexity theory and apply them in concrete program analysis.

In this work, our starting point is a previously established theory of a syntactic characterization of class NC [8]. Extending this theory, we develop an automatic static analysis that enables verifying class NC membership of recursive first-order functional programs. The analysis acts a kind of 'oracle' that responds "Yes" when it encounters an efficiently parallelizable recursive program. Our advancements include both the implementation-independent algorithmic foundations and the development of a concrete program analyzer. With the analyzer, we further gain the capability to efficiently explore the set of algorithms expressible under the restrictions imposed by the characterization. The correctness of the mathematical theory follows directly from the prior result [8, Thm. 1]. During concrete program analysis, an acceptance decision reduces to an SMT problem where a positive verdict requires confirmation from a solver.

2 Arboreal and fraternal programs

We consider programs written in a generic first-order functional programming language. The vocabulary $\Sigma = \{\mathrm{Var}, \mathrm{Cns}, \mathrm{Op}, \mathrm{Fun}\}$ consists of four disjoint sets of *symbols*: variables, construc-

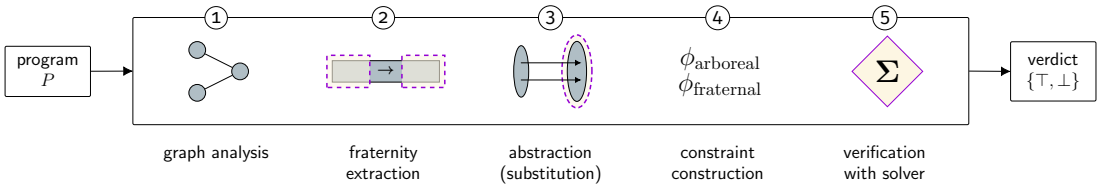


Figure 1: The analysis procedure for determining class NC membership of program P.

tors, operators, and functions. A *program* is a list of *rules* where a rule consists of a function, patterns (arguments), and an expression, written $f(p_1, \ldots, p_n) \to e$.

We will use two syntactic restrictions as the complexity measures to relate recursive functional programs to class NC. The first measure is *arboreality*, a restriction on *recursion depth*. Arboreality is concerned with recursive data and function calls, and it ensures that the number of successive recursive calls is bounded logarithmically in the input size. The second measure is *fraternality*, a restriction on *output data size*. Fraternality ensures that the size of computed values is bounded polynomially in the inputs. In reference to the classical NC definition, arboreality encodes the logarithmic time criterion and fraternality the polynomial processor criterion. The program analysis goal then becomes to determine whether a program satisfies these two complexity measures.

3 Automatic program analysis for parallelizability

Our program analysis takes as input a functional program, i.e., a list of rules; and returns as output a Boolean verdict where a positive response confirms membership in class NC. Figure 1 illustrates the described analysis workflow.

1. *Graph analysis.* From the rules, we generate graphs that carry information about recursive calls and data dependence between rules.

2. *Fraternity extraction.* From the rules and graphs, we derive fraternities. A *fraternity* is a meta-construct that represents a recursion and its surrounding context.[1] The rest of the analysis focuses exclusively on fraternities.

3. *Symbol abstraction.* From the fraternities, we collect symbols (variables, constructors, operators, and function calls). We then infer substitutions that enable translating syntactic symbols to numerically evaluatable expressions. This is an abstraction step that overestimates the program's true runtime behavior.

4. *Constraint construction.* From the fraternities and symbolic abstractions, we construct verification conditions. For arboreality, we establish that the program cannot violate logarithmic recursion depth via proof by unsatisfiability. The solver's task is to find a violating counterexample, and if no input recurses beyond logarithmic depth, the criterion is satisfied. For fraternality, we infer upper bounds on the output sizes of symbols. Showing that there exists an upper bound of acceptable polynomial shape certifies that the symbol output size is also necessarily acceptable.

[1] Be careful to avoid confusing *fraternity* (a meta-construct of rules) and *fraternality* (a complexity measure).

5. *Verification.* Using the verification conditions, we evaluate arboreality and fraternality. The analysis verdict is positive if both criteria pass the verifier. On rejection, the verifier will produce a counterexample that identifies the rules and symbols involved in the failure.

Example: parallelizable bit adding The Kernighan's bit counter—written `x & (x - 1)` in C language [7, p. 48]—is a clever way to count the number of set bits in a bitvector. Each iteration clears the least significant bit. If the bitvector has regions of contiguous zeros, the counter jumps over the zeros. We can write a similarly inspired recurrence in functional style as

```
bitseq(i(x), n) → if(no1(x), n, bitseq(and(x, pred(x)), succ(n)))
```

The `i` is a constructor and symbols `and` and `pred` are functions over bitvectors. The operator `no1` checks for the base case where the input consists entirely of zeros and `succ` increments the counter value `n`. As part of the analysis, we confirm that the output data sizes of the symbols are bounded by polynomials, thus the program is fraternal. However, the program is not arboreal. The problem is that the expression `and(x, pred(x))` is too weak; it removes one set bit at each round of recursion. In the worst case, the recursion depth is equal to the input length. The analyzer thus rejects the program, flagging an issue with arboreality.

Counting the number of set bits is however the canonical example of efficient parallel computation; therefore, an acceptable solution is achievable. To satisfy the arboreal criteria, we must find a bit counting routine that halves the input at each recursive round. We propose a program variant in the style of a summation tree [2]:

```
bitpar(i(x)) → add(bitpar(fh(i(x))), bitpar(sh(i(x))))
bitpar(1(ϵ)) → 1(ϵ)
bitpar(0(ϵ)) → 0(ϵ)
```

Function `fh` (first half) computes the leftmost $\lfloor |x|/2 \rfloor$ bits of `x`, and `sh` computes the rightmost $\lceil |x|/2 \rceil$ bits of `x`. The recursion terminates at the base cases, `1`(ϵ) and `0`(ϵ). Operator `add` is a binary associative operation that combines the partial counts. We can now prove arboreality by showing there exists no input that can violate logarithmic recursion depth. The program is arboreal and fraternal and thus accepted by the verifier.

4 Future prospects and opportunities

Currently, we have completed designing the program analysis and developed a concrete analyzer that implements the analysis. Using the analyzer, we have performed experiments to demonstrate that the analysis is expressive enough to delineate a broad class of algorithms in NC. Among the algorithms confirmed acceptable are, e.g., matrix multiply, bit-parity, associative bitvector operations, and ceiling log. We are actively exploring further the expressive capabilities.

In addition to the verdict, our analysis delivers three complementary benefits. First, it makes explicit syntactical constructs that are favorable for creating parallelization potential. Especially for optimizers and rewrite systems, awareness of specific favorable syntactical constructs is valuable heuristic information, as in [12, 5, 1] for example. Second, an instrumented analyzer can serve as a goal-state checker in program synthesis. However, such a checker must be practically efficient enough to keep verification overhead manageable. Our preliminary experiments give evidence of such practicality, as many analyses conclude in mere milliseconds. Finally, if a program is rejected, it is possible to locate precisely the failure-inducing fragment. This is because the analysis generates a counterexample if arboreality or fraternality is violated. The

feedback localizes the issue and guides refactoring effort to find a satisfactory program variant – as in the bit-adding example.

We are optimistic that many directions of future research can follow from here. We are particularly interested in investigating applications in compile-time optimization or program synthesis by having the NC membership verifier serve as an intermediate goal-state checker.

References

[1] Clément Aubert, Thomas Rubiano, Neea Rusch, and Thomas Seiller. Distributing and parallelizing non-canonical loops. In *Verification, Model Checking, and Abstract Interpretation*, volume 13881 of *LNCS*, pages 1–24, Cham., 2023. Springer. doi:10.1007/978-3-031-24950-1˙1.

[2] Guillaume Bonfante, Jean-Yves Marion, and Romain Péchoux. A characterization of alternating log time by first order functional programs. In Miki Hermann and Andrei Voronkov, editors, *Logic for Programming, Artificial Intelligence, and Reasoning*, volume 4246 of *LNCS*, pages 90–104, Berlin, Heidelberg, 2006. Springer. doi:10.1007/11916277˙7.

[3] Stephen A. Cook. An overview of computational complexity. *Communications of the ACM*, 26(6):400–408, 6 1983. doi:10.1145/358141.358144.

[4] Stephen A. Cook. A taxonomy of problems with fast parallel algorithms. *Information and Control*, 64(1-3):2–22, 3 1985. doi:10.1016/s0019-9958(85)80041-3.

[5] Anastasia Isychev and Eva Darulova. Scaling up roundoff analysis of functional data structure programs. In Manuel V. Hermenegildo and José F. Morales, editors, *Static Analysis*, volume 14284 of *LNCS*, pages 371–402, Cham., 2023. Springer. doi:10.1007/978-3-031-44245-2˙17.

[6] Joseph JáJá. *An Introduction to Parallel Algorithms*. Addison-Wesley Publishing Company, Reading, MA, 1st edition, 1992.

[7] Brian W. Kernighan and Dennis M. Ritchie. *C Programming Language*. Pearson, Lebanon, IN, USA, 2nd edition, 1988.

[8] Jean-Yves Marion and Romain Péchoux. A characterization of NC^k by first order functional programs. In Manindra Agrawal, Dingzhu Du, Zhenhua Duan, and Angsheng Li, editors, *Theory and Applications of Models of Computation*, volume 4978 of *LNCS*, pages 136–147, Berlin, Heidelberg, 2008. Springer. doi:10.1007/978-3-540-79228-4˙12.

[9] Nicholas Pippenger. On simultaneous resource bounds. In *20th Annual Symposium on Foundations of Computer Science*, sfcs 1979, pages 307–311, New York, NY, USA, 1979. IEEE. doi:10.1109/sfcs.1979.29.

[10] Walter L. Ruzzo. On uniform circuit complexity. *Journal of Computer and System Sciences*, 22(3):365–383, June 1981. doi:10.1016/0022-0000(81)90038-6.

[11] Yossi Shiloach and Uzi Vishkin. Finding the maximum, merging and sorting in a parallel computation model. In W. Brauer, P. Brinch Hansen, D. Gries, C. Moler, G. Seegmüller, J. Stoer, N. Wirth, and Wolfgang Händler, editors, *Conpar 81*, pages 314–327, Berlin, Heidelberg, 1981. Springer. doi:10.1007/BFb0105127.

[12] Michel Steuwer, Toomas Remmelg, and Christophe Dubach. Lift: A functional data-parallel ir for high-performance gpu code generation. In Vijay Janapa Reddi, Aaron Smith, and Lingjia Tang, editors, *2017 IEEE/ACM International Symposium on Code Generation and Optimization*, CGO, pages 74–85, Piscataway, NJ, USA, 2017. IEEE. doi:10.1109/cgo.2017.7863730.